

**UNIVERSIDAD DE
SAN BUENAVENTURA
CALI**



Facultad de Ingeniería | Programa de
Ingeniería de Sistemas
Diseño Detallado de Software

Proyecto Corte 1
Estanterías Inteligentes INC

Ana María Bautista Rodríguez 95429

Juan Andrés Veintinilla 124596

2026-1

-
- 1 Introducción
 2. Objetivo General
 - 2.1 Objetivos Específicos
 - 4 Diagrama de Contexto General
 - 5 Diagrama de Contexto de Funcionalidad
 - 5.1 Localización y Visualización de Productos Disponibles
 - 5.2 Resurtido Anticipado de Productos
 - 5.3 Integración con identificadores Electrónicos
 - 5.4 Proceso de Compra y Pago integrado
 - 5.5 Despacho Asistido de Productos
 - 5.6 Panel Administrativo para operadores de Inventario
 - 5.7 Propuesta de Valor
 - 6 Diagrama de Contenedores C4
 - 7 Diagrama de componentes para el backend de la aplicación
 - 8 Patrones de Diseño
 - 8.1 Clasificación y Descripción de Patrones Seleccionados
 - 8.1.1 Patrón Composite (Compuesto)
 - 8.1.2 Patrón Visitor (Visitante)
 - 8.1.3 Patrón Adapter (Adaptador)
 - 8.1.4 Patrón Observer (Observador)
 - 8.1.5 Tabla de Organización de Patrones
 - 9 Prototipado y Simulación
 - 9.1 Prototipado con Patrón “Composite”
 - 9.1.1 Solución Prototipo “Composite”
 - 9.1.1.1 InventarioComponent
 - 9.1.1.2 Empresa (Root)
 - 9.1.1.3 Bodega (Nodo Compuesto / Rama)
 - 9.1.1.4 Estantería (Nodo / Rama)
 - 9.1.1.5 Producto (Leaf / Hoja)
 - 9.2 Prototipado con Patrón “Visitor”
 - 9.2.1 Solución Prototipado “Visitor”
 - 9.2.1.1 InventarioVisitor (Interfaz)
 - 9.2.1.2 Producto (Clase)
 - 9.2.1.3 SensorIoT (Nueva Funcionalidad)
 - 9.2.1.4 ReporteStock (Nueva Funcionalidad)
 - 10 Reflexión y Evaluación de los Patrones
 - 10.1 Reflexión y Evaluación de los Patrones
 - 10.2 Uso de IA para prototipado
-

1 Introducción

El presente documento detalla el diseño de software para el sistema "Estanterías Inteligentes INC", una solución orientada a la optimización de inventarios mediante la convergencia de tecnologías IoT (Internet de las Cosas) e Inteligencia Artificial. A diferencia de los métodos de control de stock convencionales, este sistema propone un enfoque proactivo y predictivo, reduciendo el error humano y maximizando la eficiencia operativa en sectores críticos como salud y retail.

2 Objetivo General

Desarrollar un sistema inteligente de gestión de inventarios en tiempo real, mediante la integración de software especializado, sensores IoT y algoritmos de inteligencia artificial, con el fin de optimizar la operación logística y anticipar necesidades de reabastecimiento.

2.1 Objetivos Específicos

- Analizar el problema logístico de inventarios en distintos sectores (retail, dark stores, hospitales, autopartes), identificando las limitaciones de los métodos tradicionales y justificando la necesidad de un sistema inteligente.
- Diseñar e implementar el software de gestión predictiva, integrando algoritmos de inteligencia artificial y dashboards en la nube para procesar datos en tiempo real y anticipar necesidades de reabastecimiento.
- Integrar hardware IoT en estanterías inteligentes, mediante sensores de peso, RFID y sistemas de luces Pick-to-Light, asegurando la comunicación con el software a través de redes inalámbricas industriales.
- Validar la eficiencia e innovación del sistema, midiendo mejoras en tiempos de despacho, reducción de pérdidas y explorando la incorporación futura de tecnologías emergentes como visión artificial, blockchain o realidad aumentada

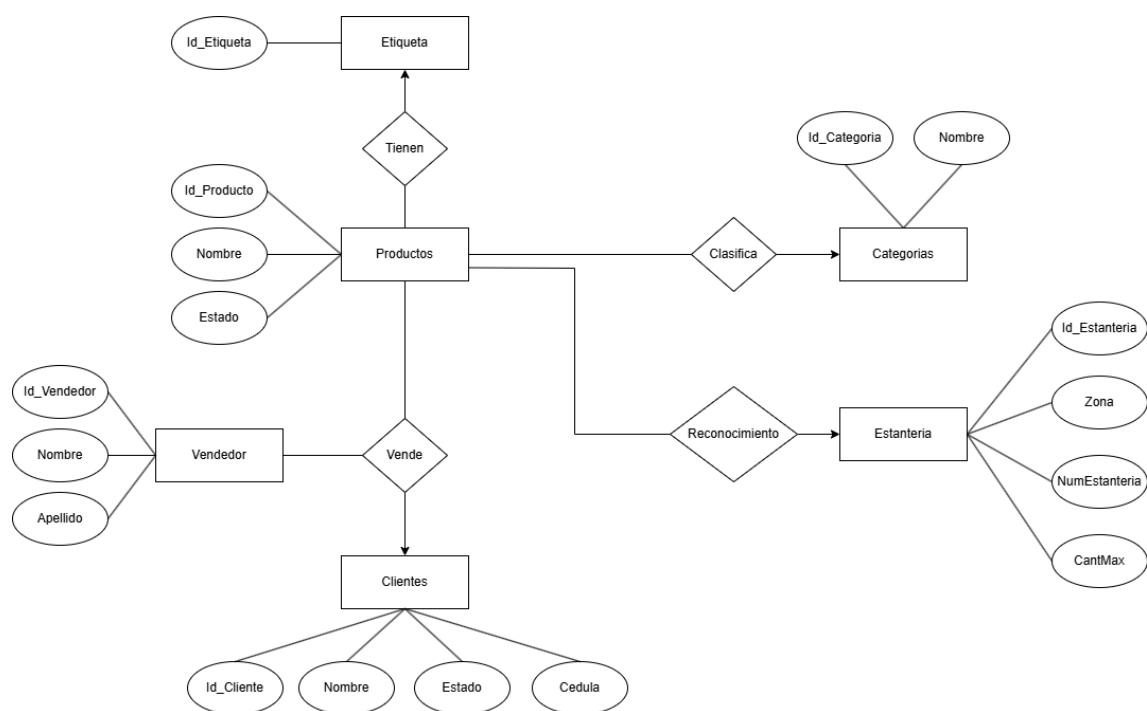
3 Modelo Canvas con Redacción Textual del Modelo

El modelo de negocio de **Estanterías Inteligentes INC** se centra en la eficiencia logística. Nuestra **Propuesta de Valor** es la eliminación de quiebres de stock y la automatización del inventario físico. Los **Segmentos de Cliente** incluyen hospitales (insumos médicos), dark stores y bodegas de autopartes. Nuestros **Canales** de distribución son la venta directa B2B y consultorías de implementación. Las **Fuentes de Ingresos** provienen de la venta del hardware (estanterías), licenciamiento del software (SaaS) y mantenimiento técnico.

SOCIOS CLAVE 1) Proveedores de Sensores y componentes Electrónicos así mismo alianzas con empresas locales de metalmecánica para integrar la tecnología en sus muebles) 2) Empresas de Software ERP (Oracle, SAP, AWS, Google Cloud) 3) Universidades o centros de Investigación en IA	ACTIVIDADES CLAVE 1) Desarrollo del software (Programación del backend y algoritmos predictivos) 2) Integración con plataformas de gestión(Ensamblaje de circuitos en estanterías metálicas.) 3) Producción y ensamblaje(Puesta a punto en la bodega del cliente.) 4)Gestión de la base de datos en la nube. RECURSOS CLAVE 1) Equipo de desarrolladores soft/hardware(Desarrolladores Full-stack, Ingenieros de Datos (para la predicción) y técnicos instaladores.) 2) Sensores IoT (Sensores de peso, etiquetas RFID, luces LED direccionables.) 3) Algoritmos de predicción (IA) 4) Red robusta (Zigbee/Wi-Fi industrial)	PROPUESTA DE VALOR 1. Eficiencia Operativa: Reducción del tiempo de despacho (picking) hasta un 50% mediante sistema de luces guía (Pick-to-Light). 2. Inventario en Tiempo Real: Eliminación del "conteo manual" y reducción de pérdidas por robo hormiga (común en bodegas locales). por medio de etiquetas de seguridad. 3. Predicción Inteligente: Algoritmos que sugieren reabastecimiento antes de que se agote el producto, crucial para temporadas altas (Día sin IVA, Navidad).	RELACION CON LOS CLIENTES 1) Soporte técnico 24/7 / 2) Contratos de Mantenimiento y actualización del software como socios CANALES 1. Venta Consultiva Directa: Visitas a gerentes de logística y operaciones. 2. Ferias Logísticas: Participación en eventos como Logistec en Bogotá. 3. Alianzas con proveedores de ERP: Integradores de SAP, Oracle o Siigo que ofrezcan esto como un módulo de hardware adicional.	SEGMENTO DE CLIENTES 1)Centros de Distribución (CENDIS): De almacenes de cadenas retail (ej. Éxito, D1, Olímpica). 2. Dark Stores: Bodegas ocultas para entregas rápidas (ej. Rappi Turbo, aplicaciones de delivery). 3. Farmacéuticas y Hospitales: Donde el control de inventario y fechas de vencimiento es crítico. 4. Empresas de Autopartes: Manejo de miles de referencias pequeñas.
ESTRUCTURA DE COSTES 1. I+D: Salarios de ingeniería y desarrollo. 2. Hardware: Importación de microcontroladores, sensores y luces LED. 3. Infraestructura Nube: Costos variables según el volumen de datos procesados. 4. Comercial: Costos de adquisición de clientes (CAC) y marketing B2B.		FUENTE DE INGRESOS 1) Venta del Hardware (Estanterías): Costo por metro lineal de estantería instalada y sensores. 2) Licencias por uso del software de inventario predictivo y dashboard en la nube 3) Servicio de instalación y personalización 4) Servicio de Mantenimiento y actualización anuales para calibrar los sensores		

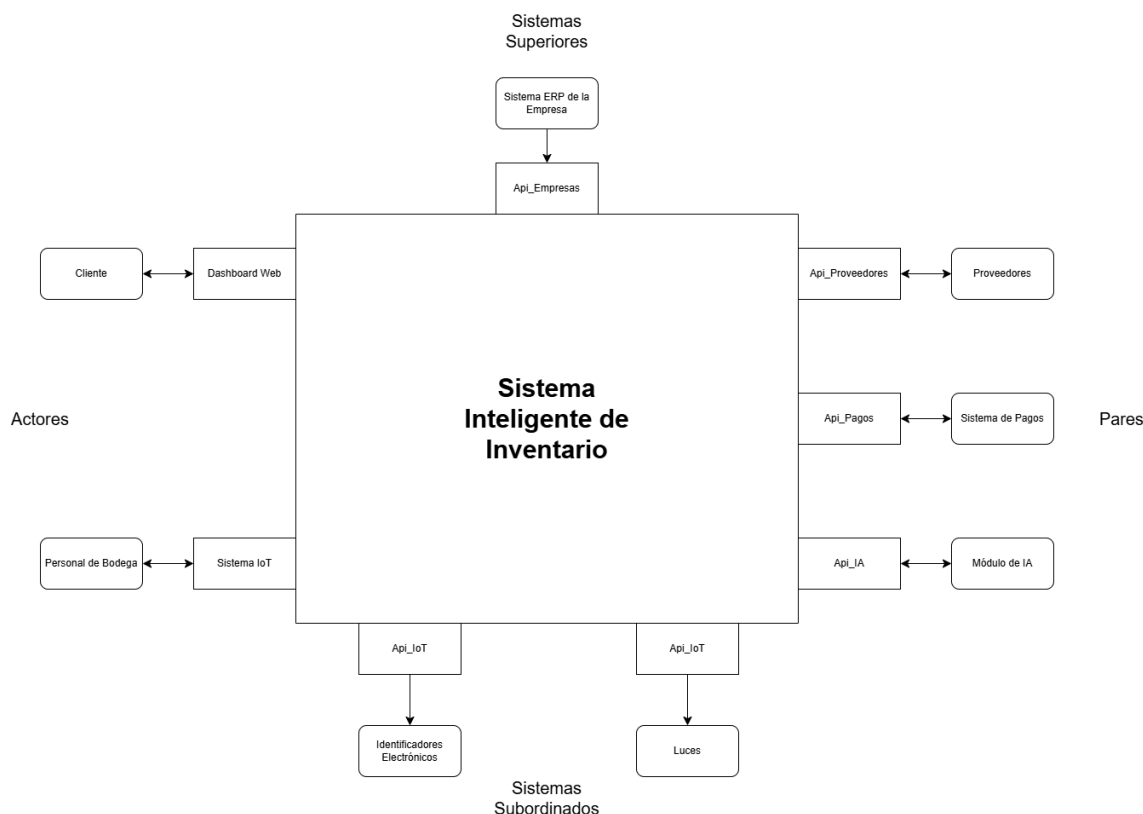
4. Modelo de Datos del Sistema: Modelo Entidad Relación

El sistema se sustenta en una base de datos relacional que gestiona entidades como productos, etiquetas, vendedor, categorías y estantería. La relación cardinal destaca que una estantería puede contener múltiples productos, y cada producto es monitoreado por sensores específicos cuyos datos se almacenan históricamente para alimentar el modelo de IA.



4 Diagrama de Contexto General

El diagrama de contexto representa el nivel más alto de abstracción del sistema Estanterías Inteligentes INC, delimitando las fronteras entre el software y los actores (entidades externas) que interactúan con él. Este modelo permite visualizar el flujo de información de entrada y salida sin entrar en detalles técnicos.



4.1 Descripción de Entidades Externas

En el diseño de Estanterías Inteligentes INC, el sistema interactúa con un ecosistema de APIs externas que permiten la escalabilidad y la especialización de funciones. A continuación se detallan estas entidades:

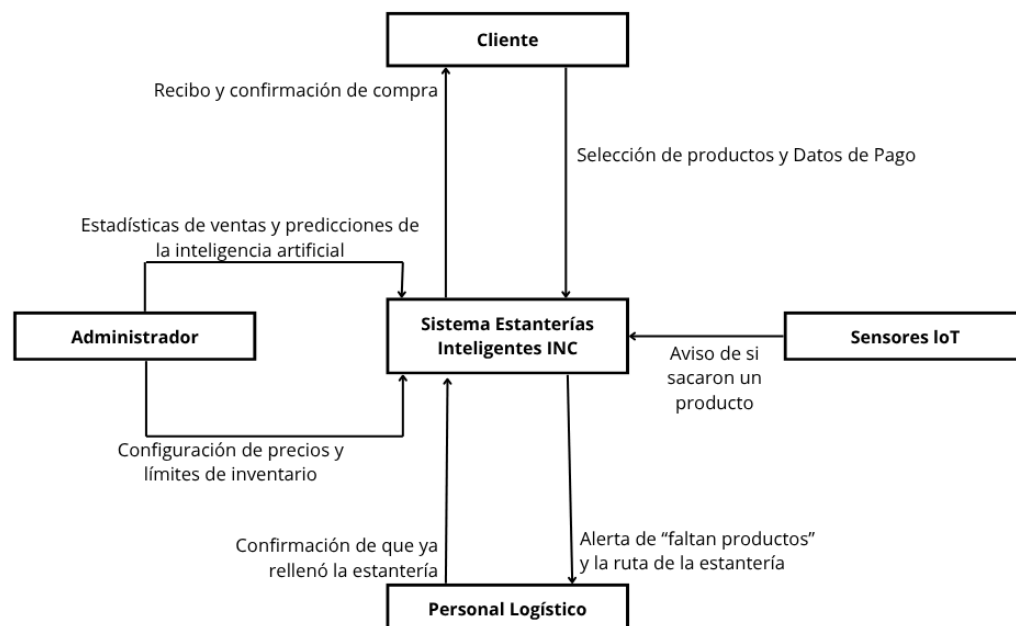
- **API_Empresas:** Es la entidad encargada de proveer y validar la información legal y organizacional de las empresas que contratan el servicio. Permite la gestión de múltiples sedes o sucursales bajo un mismo contrato maestro, facilitando la segregación de datos por organización.
- **API_Clientes:** Gestiona el perfilamiento de los usuarios finales y clientes del sistema. Se encarga de la autenticación, niveles de acceso y preferencias de usuario, asegurando que la experiencia en el dashboard sea personalizada según el rol del cliente (ej. Jefe de planta vs. Operador).
- **API_Proveedores:** Esta entidad externa centraliza la comunicación con los suministradores de productos. Facilita la sincronización de catálogos y, en etapas

avanzadas, permite la emisión automática de órdenes de compra cuando el sistema detecta que el stock está por debajo del umbral crítico.

- **API_Pagos:** Actúa como la pasarela de servicios financieros. Es la responsable de procesar las transacciones económicas derivadas de las compras directas o del pago de suscripción del servicio SaaS, garantizando estándares de seguridad bancaria y cifrado de datos.
- **API_Bodega:** Se encarga de la lógica de ubicación macro. Mientras el sistema gestiona la estantería específica, esta API comunica el estado del inventario con el sistema de gestión de almacenes (WMS) general de la empresa, permitiendo la trazabilidad del producto desde que ingresa al muelle de carga hasta que se coloca en la estantería inteligente.
- **API_IA (Inteligencia Artificial):** Es el motor de procesamiento avanzado. Recibe los datos históricos de consumo enviados por el sistema y retorna predicciones probabilísticas sobre el agotamiento de stock. Utiliza modelos de aprendizaje supervisado para identificar patrones estacionales y tendencias de consumo.
- **API_IoT (Internet of Things):** Es la capa de abstracción para el hardware. Esta entidad gestiona la comunicación con los protocolos industriales (como MQTT), procesando las señales crudas de los sensores de peso y etiquetas RFID para convertirlas en datos estructurados que el backend del software pueda interpretar.

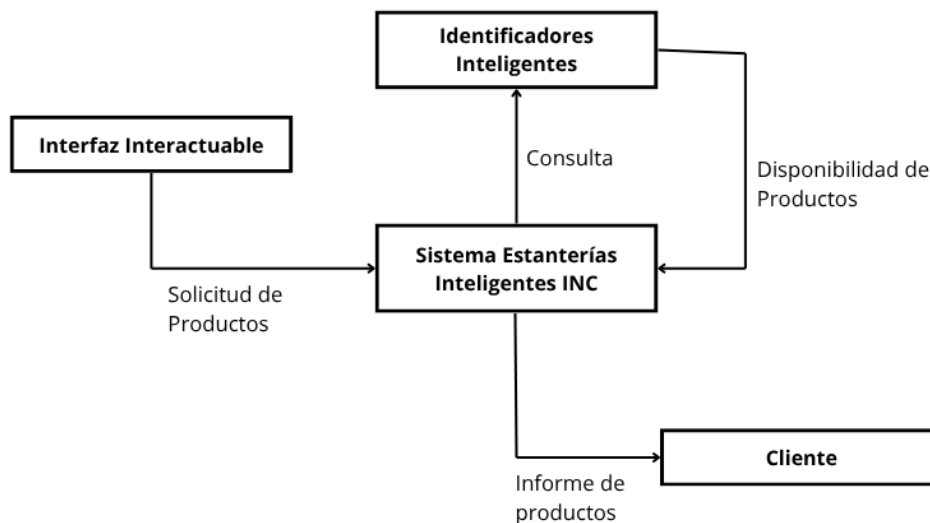
5 Diagrama de Contexto de Funcionalidad

El diagrama de contexto de funcionalidad detalla las interacciones operativas entre las entidades externas y los procesos core del sistema Estanterías Inteligentes INC. A diferencia del diagrama general, aquí se especifican los flujos de datos exactos que permiten la gestión automatizada del inventario.



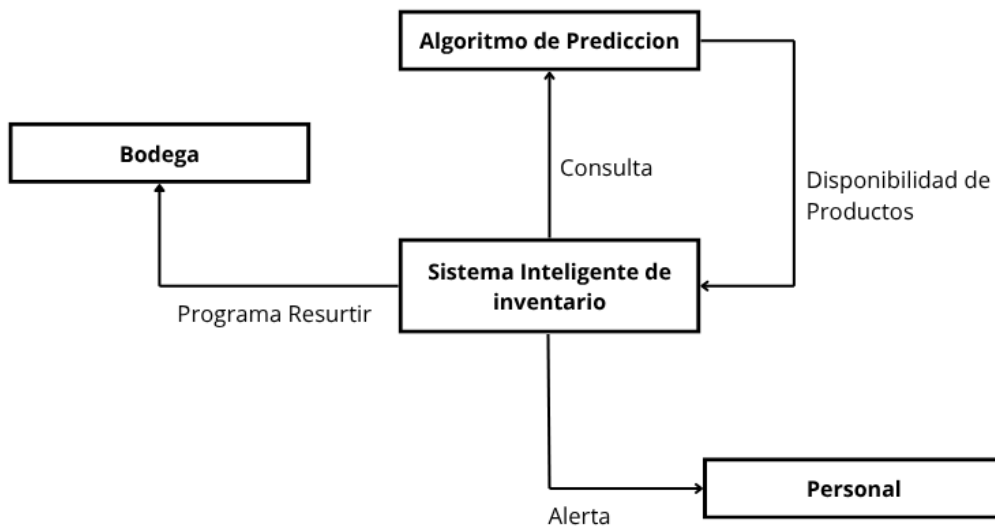
5.1 Localización y Visualización de Productos Disponibles

Esta funcionalidad permite al usuario (operador o administrador) conocer la ubicación exacta y el estado de stock de cualquier artículo dentro del ecosistema de estanterías inteligentes. Es el puente entre el mundo digital y el espacio físico del almacén.



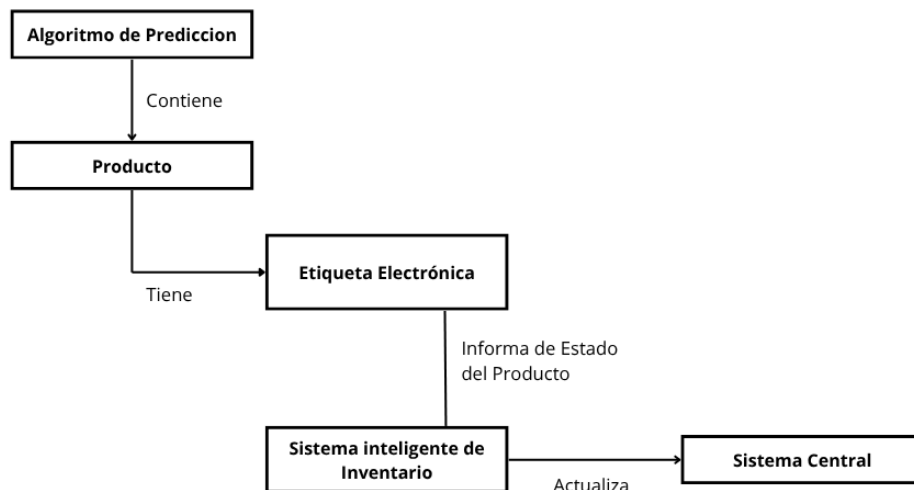
5.2 Resurtido Anticipado de Productos

Esta funcionalidad permite al usuario conocer en tiempo real la ubicación exacta y la disponibilidad de los productos dentro del sistema de estanterías inteligentes. A través de la consulta a identificadores inteligentes, el sistema integra información física y digital para generar informes detallados del estado del inventario, facilitando la búsqueda y gestión eficiente de los productos.



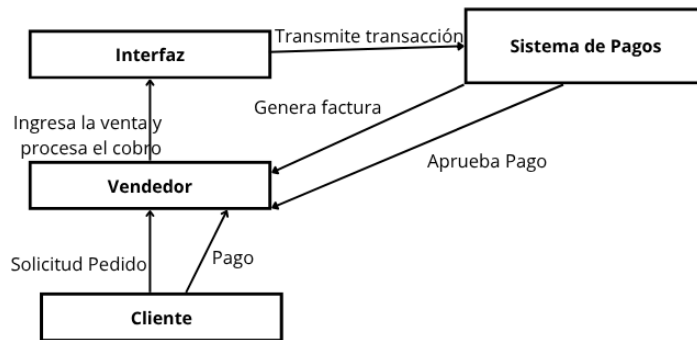
5.3 Integración con identificadores Electrónicos

Esta funcionalidad permite la captura y actualización automática del estado de los productos mediante el uso de etiquetas electrónicas. Cada producto está asociado a un identificador que transmite información en tiempo real al sistema de inventario, el cual procesa y sincroniza estos datos con el sistema central, garantizando precisión, trazabilidad y actualización constante del inventario.



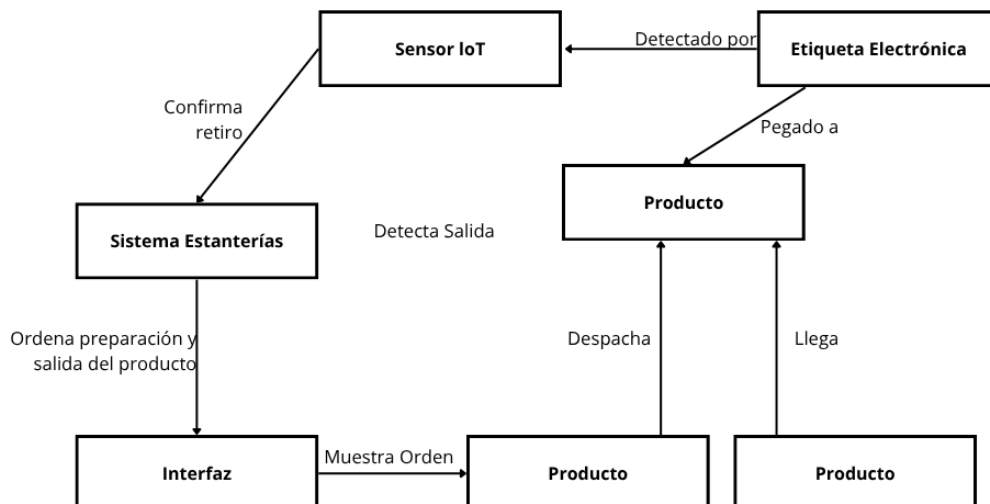
5.4 Proceso de Compra y Pago integrado

Esta funcionalidad articula el ciclo completo de adquisición dentro del ecosistema de estanterías inteligentes, desde la solicitud inicial del cliente hasta la confirmación financiera final. El flujo conecta al vendedor con la interfaz digital y el sistema de pagos, garantizando que cada transacción sea registrada, validada y respaldada con comprobantes electrónicos.



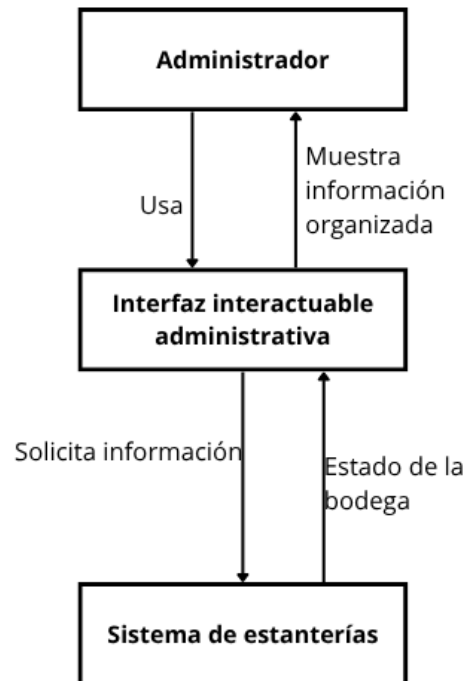
5.5 Despacho Asistido de Productos

Esta funcionalidad garantiza que el proceso de salida de mercancías se realice de manera controlada y eficiente, integrando sensores IoT y etiquetas electrónicas para validar cada movimiento. El sistema de estanterías actúa como coordinador central, confirmando el retiro de productos y ordenando su preparación a través de la interfaz digital.



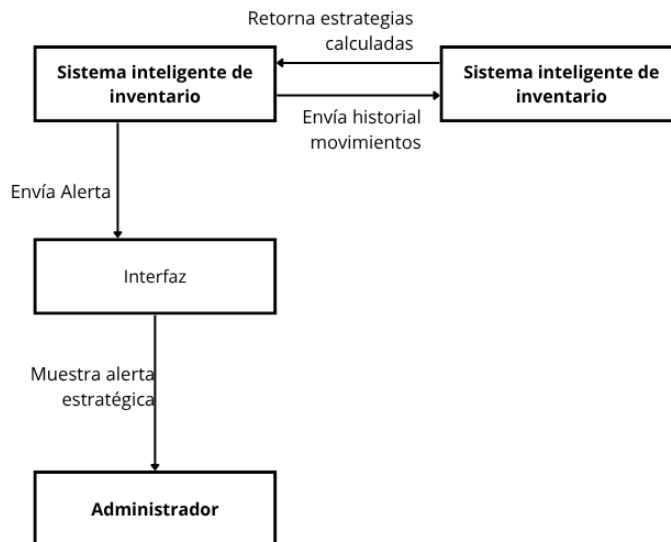
5.6 Panel Administrativo para operadores de Inventario

Esta funcionalidad brinda al administrador una interfaz interactiva que centraliza la gestión del estado de la bodega y la organización del inventario. El panel actúa como un puente entre el sistema de estanterías inteligentes y el usuario, ofreciendo información clara y estructurada para la toma de decisiones.



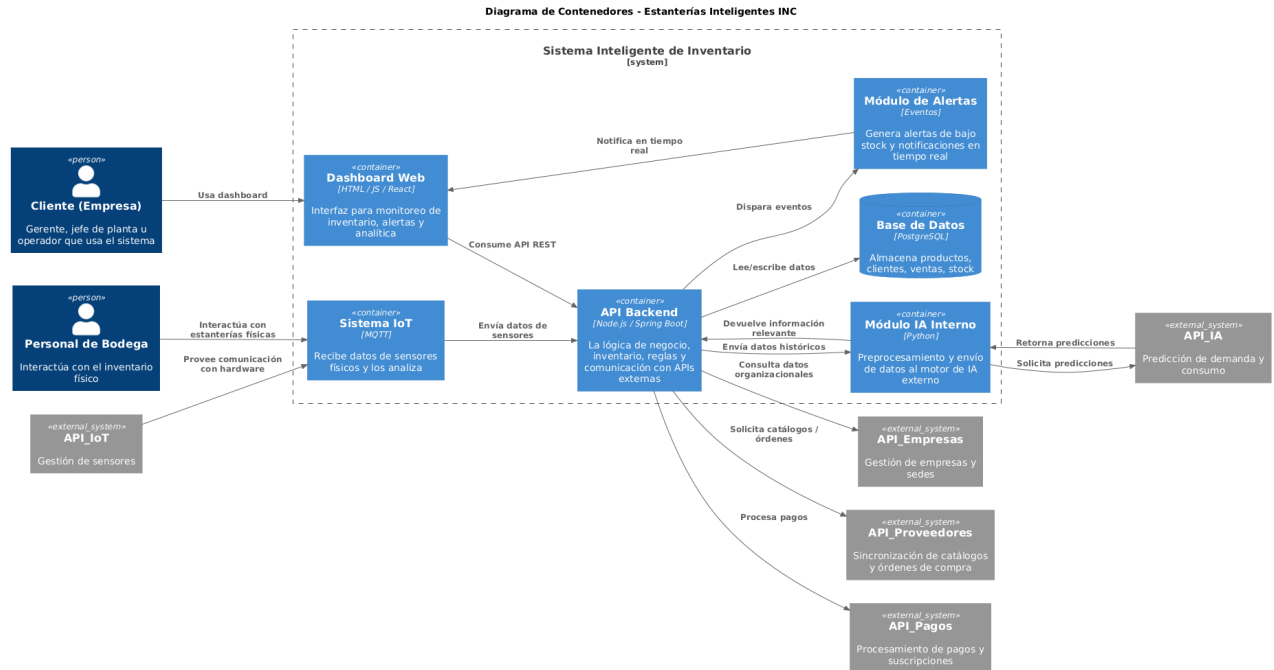
5.7 Propuesta de Valor

Esta funcionalidad representa la capacidad estratégica del sistema inteligente de inventario para transformar datos históricos en acciones concretas. El sistema analiza los movimientos registrados y, a partir de ellos, calcula estrategias de optimización que se convierten en alertas estratégicas para el administrador.



6 Diagrama de Contenedores C4:

Diagrama de Contenedores Principal donde se muestra el flujo de la aplicación



Flujo del Diagrama de Contenedores C4:

❖ Actores:

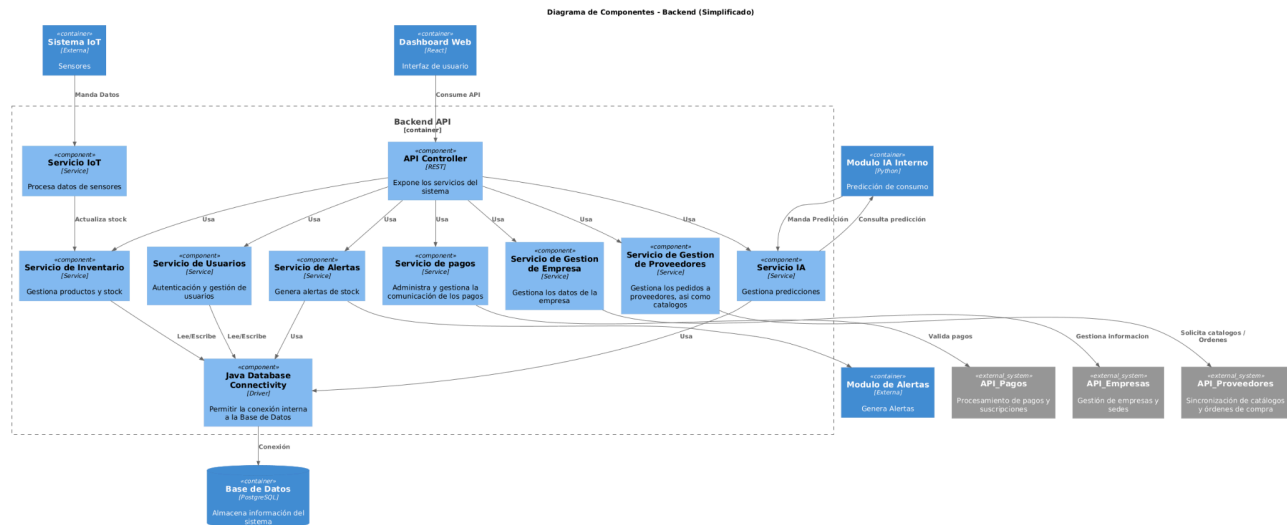
- **Cliente (Empresa):** Encargado de manejar el dashboard web que la aplicación provee para la realización de todas las actividades pertinentes del propio sistema.
- **Personal de Bodega:** Actor que interactúa con nuestro sistema al usar las repisas con sensores IoT que se comunican directamente al backend del sistema para la actualización a tiempo real de los inventarios y otros aspectos del sistema.

❖ Backend:

- **DashBoard Web:** Dashboard que contiene toda la información relevante para el uso del cliente
- **Sistema IoT:** Módulo dentro del sistema que está conectado con el backend para transmitir la información procesada de los elementos IoT.
- **Base de Datos:** la cual contiene la información de los clientes, datos de productos, entre otras cosas
- **Módulo de Alertas,** Un módulo aparte dentro del sistema que se encarga de enviar al dashboard del cliente notificaciones de falta de stock y demás información. Todo esto se logra gracias a que el backend le suministra la información necesaria para mandar las alertas
- **Módulo de IA interno:** Módulo encargado de procesar y recibir información de la Api_IA, la cual es entrenada y regresa información sobre predicciones de demanda y consumo.

-
- 1) **Api_Empresa:** Sistema Externo que se comunica con el backend para traer información relevante de la empresa que esté recibiendo el servicio de la aplicación, se sacan validaciones entre otras cosas.
 - 2) **Api_Proveedores:** Sistema externo el cual el backend requiere para pedir catálogos de productos que la empresa puede llegar a pedir para los restocks
 - 3) **Api_Pagos:** Sistema externo encargado de gestionar las comunicaciones financieras del sistema con otras partes del backend.

7 Diagrama de componentes para el backend de la aplicación



Flujo del Diagrama de Componentes para el Backend de la aplicación:

❖ Contenedores:

- 1) **Sistema IoT:** El contenedor IoT se conecta con un servicio del backend llamado “Servicio IoT”, el cual a su vez manda información directamente hacia otro servicio de inventarios para acabar dentro la Base de datos de la aplicación.
- 2) **DashBoard Web:** Éste contenedor recibe toda la información del backend, por lo cual se conecta directamente al mismo dicho antes para usar todos los servicios que puede proveer.
- 3) **Módulo de Alertas:** El Api Controller del backend se comunica con un servicio de Alertas para mandar las notificaciones de diferentes aspectos de la base de datos hacia el módulo de alertas.
- 4) **Base de Datos:** Los servicios del backend que tienen una relación con cualquier contenedor del sistema acaban por conectarse directamente con la Base de Datos para mandar y obtener información, y para ello deben primero pasar por el servicio de conexión de la base de datos llamada “Java Database Connectivity”
- 5) **Módulo IA Interno:** Se conecta al backend mediante el servicio de IA, la cual pide y recibe predicciones de demanda y oferta para suministrar a los demás componentes dentro del backend.

● Externos:

- 1) **Api_Empresas:** El backend se conecta con éste contenedor externo mediante el uso de un servicio de empresas, con ello se busca información relevante, así como credenciales necesarias de la empresa la cual está trabajando con la aplicación.
- 2) **Api_Proveedores:** La api Controller del backend maneja la comunicación con los proveedores mediante el uso de un servicio de proveedores, específico para la pedida de catálogos, productos, entre otras cosas que puede darnos el contenedor de Api_Proveedores
- 3) **Api_Pagos:** Para la realización de transacciones y otros procesos monetarios, el backend se conecta con los sistemas de pago mediante un servicio de pagos, que ayuda a realizar las acciones antes descritas.

8 Patrones de Diseño

El sistema de Inventario de Estanterías Inteligentes requiere una solución arquitectónica robusta que permita gestionar jerarquías complejas de datos, integrar hardware heterogéneo y mantener la extensibilidad a lo largo del tiempo. Para dar respuesta a estos retos se seleccionaron cuatro patrones de diseño canónicos, clasificados...

8.1 Clasificación y descripción de patrones seleccionados

Patrón	Clasificación	Propósito en el Sistema
Composite	Estructural	Representar la jerarquía física del inventario (Empresa > Bodega > Estantería > Producto) de forma uniforme para realizar consultas masivas.
Visitor	Conductual	Ejecutar operaciones como “Reporte inventario”, “Cálculo del valor Total” o “Configuración de sensores” sobre la jerarquía sin modificar las clases de hardware.
Adapter	Estructural	Integrar sensores de diferentes fabricantes y tecnologías bajo una interfaz estándar, permitiendo que el sistema procese lecturas de peso, presión o RFID sin modificar la lógica principal del backend.
Observer	Comportamiento	Notificar automáticamente a múltiples módulos del sistema cuando se detecta un cambio en el inventario, permitiendo actualizar el dashboard, generar alertas de stock y registrar eventos sin acoplar directamente los sensores con cada servicio.

8.1.1 Patrón Composite (Compuesto)

- **Problema:** El sistema sí o sí debe de manejar un sistema de jerarquía compleja. Porque una empresa puede tener múltiples bodegas, cada bodega tiene varias estanterías, y cada estantería tiene diversos productos. Si queremos saber el stock total de una empresa o de una sola estantería, el código puede volverse redundante con múltiples ciclos for.
- El patrón Composite nos permite tratar a los objetos individuales (Productos) y a los contenedores (Estanterías/Bodegas) de la misma forma a través de una interfaz llamada *ComponenteInventario*
- Para implementarlo en el sistema podría ser de la siguiente manera:
 - **Componente (Interfaz):** Define el método mostrarDetalle() y obtenerStock()
 - **Hoja (Producto):** Es el nivel más bajo, retorna su propio stock.
 - **Compuesto (Estantería/Bodega):** Contiene una lista de componentes. Cuando se le pide el stock, suma el stock de todos sus hijos automáticamente.

8.1.2 Patrón Visitor (Visitante)

- **Problema:** Queremos agregar más funcionalidades constantemente, cómo por ejemplo “Calculador de Alertas” o “Generador de Reportes PDF”, pero no queremos estar modificando cada rato las clases de Estantería o Producto cada vez que se nos ocurra una nueva función, para no romper ese principio de responsabilidad única.
- Implementación en el sistema:
 - **Visitor(Interfaz):** Define métodos como visitarProducto(Producto p) y visitarEstanterías(Estanteríe)
 - **Visitante Concreto (ReporteStock):** recorre toda la jerarquía acumulando datos para generar un informe.
 - **Visitante Concreto (AlertaMantenimiento):** Revisa el estado de los sensores en cada estantería para reportar fallos técnicos

8.1.3 Patrón Adapter (Adaptador)

- **Problema:** Se deben integrar sensores de diferentes fabricantes y tecnologías (ej. básculas de peso análogas, sensores digitales de presión, lectores RFID de distintas marcas). Cada hardware tiene su propia interfaz de comunicación y formato de datos. Si el sistema intentara conectarse directamente a cada uno, el código del backend estaría lleno de condicionales y sería imposible de mantener.
- Implementación del sistema:
 - **Target (Interfaz Estándar):** Se define una interfaz ISensorInventario con métodos genéricos como leerDato().
 - **Adapter (Objeto a adaptar):** Son las librerías específicas de cada sensor (ej. SensorPesoPesoLux o LectorRFID_v2).
 - **Adapter (Adaptador Concreto):** Se crea una clase para cada sensor (ej. AdaptadorSensorPeso) que implementa la interfaz estándar y, por dentro, llama a los métodos complejos del fabricante. Esto permite que el sistema trate a todos los sensores de forma uniforme, sin importar su marca o modelo.

8.1.4 Patrón Observer (Observador)

- **Problema:** El inventario es dinámico cuando un producto es retirado de la estantería, el cambio de peso es detectado inmediatamente por el sensor. El sistema necesita que múltiples módulos (el Panel Administrativo, el Sistema de Alertas y el Motor de IA) se enteren de este cambio al instante para actualizar la interfaz o disparar una orden de compra, sin que el sensor tenga que preguntarles uno por uno si están interesados.
- Implementación del sistema:
 - Sujeto (Subject): La clase EstanteriaInteligente. Esta mantiene una lista de suscriptores y les notifica cuando los sensores detectan un cambio de estado.
 - Dashboard UI: Se actualiza automáticamente para mostrar el nuevo stock visualmente.
 - Sistema de Alertas: Verifica si el nuevo peso está por debajo del umbral mínimo para enviar un correo o notificación push.
 - Registro de Auditoría: Guarda el evento en la base de datos para análisis histórico.

8.1.5 Tabla de organización de patrones

Siguiendo la metodología, la siguiente tabla clasifica los patrones seleccionados según su arquitectura a la que pertenece

	Base de Datos	Aplicación	Implementación	Infraestructura
Datos / Contenido	Composite			
Manejo jerárquico de empresas, bodegas, estanterías y productos dentro del inventario		Composite		
Arquitectura	Visitor			
Incorporación futura de nuevas funcionalidades			Visitor	
Nivel de Componentes	Adapter			
Administración uniforme de distintos dispositivos IoT conectados al sistema			Adapter	
Interfaz de Usuario	Observer			
Visualización de información sobre el inventario y las alarmas inteligentes		Observer		

9 Prototipado y Simulación

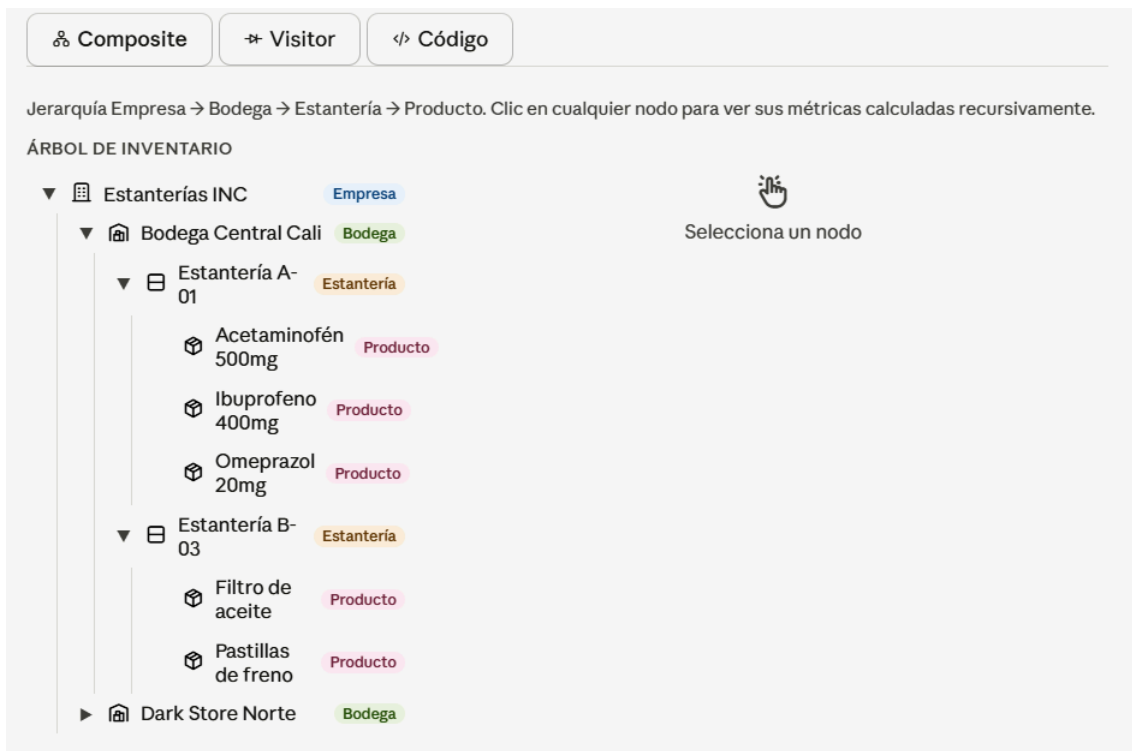
9.1 Prototipado con Patron “Composite”

Nuestro sistema de Inventario de Estanterías Inteligentes gestiona una jerarquía de cuatro niveles.

- 1) **Empresa:** Nuestro cliente corporativo, el que contrata el servicio
- 2) **Bodega:** El lugar físico que tiene la empresa para sus almacenes
- 3) **Estantería:** La unidad de almacenamiento dentro de las bodegas
- 4) **Producto:** Los ítems individuales con Stocks y precios

El problema anteriormente encontrado era la falta de una gestión jerárquica uniforme, por lo cual, sin un patrón estructural, el código para la gestión debía tratar a cada nivel con lógicas diferentes, por lo que generaba dificultades a la hora de agregar nuevos niveles.

9.1.1 Solución Prototipo “Composite”



🏠 Composite
➡ Visitor
</> Código

Jerarquía Empresa → Bodega → Estantería → Producto. Clic en cualquier nodo para ver sus métricas calculadas recursivamente.

ÁRBOL DE INVENTARIO

- ▼ 📦 Estanterías INC Empresa
- ▼ 🏠 Bodega Central Cali Bodega
 - ▼ 📦 Estantería A-01 Estantería
 - 📦 Acetaminofén 500mg Producto
 - 📦 Ibuprofeno 400mg Producto
 - 📦 Omeprazol 20mg Producto
 - ▼ 📦 Estantería B-03 Estantería
 - 📦 Filtro de aceite Producto
 - 📦 Pastillas de freno Producto
- ▶ 🏠 Dark Store Norte Bodega

📦

Estantería A-01
 Estantería · Farmacia

Unidades totales	Valor total	Productos
433	\$1.358.000	3

Alertas de stock bajo (1)

Ibuprofeno 400mg
18/30 uds

Composite

Visitor

Código

Jerarquía Empresa → Bodega → Estantería → Producto. Clic en cualquier nodo para ver sus métricas calculadas recursivamente.

ÁRBOL DE INVENTARIO

Estanterías INC

Bodega Central Cali

Estantería A-01

Acetaminofén 500mg

Ibuprofeno 400mg

Omeprazol 20mg

Estantería B-03

Filtro de aceite

Pastillas de freno

Dark Store Norte

Ibuprofeno 400mg

Producto

Unidades totales

18

Valor total

\$63.000

Precio unitario

\$3.500

SKU

FAR-002

Proveedor

LabCo

Stock mínimo

30 uds

Peso

0.18 kg

Estado:

Stock crítico

Para este prototipo se realizó lo siguiente: Implementar la jerarquía Empresa → Bodega → Estantería → Producto. Todos los nodos implementan la misma interfaz “InventarioComponent”, lo que resuelve el problema anteriormente identificado: el manejo jerárquico uniforme para nuestro sistema, dejando espacio para inclusión de nuevos niveles si fuera necesario. Al hacer clic en cualquier nodo, las métricas (unidades, valor total, alertas) se calculan recursivamente delegando a los hijos — un nodo compuesto no sabe si sus hijos son hojas u otros compuestos, simplemente les delega.

2026

9.1.1.1 InventarioComponent

Esta parte del código dentro del patrón de Composite se le conoce como: “Component” o “Interfaz Común”. Es la interfaz o clase abstracta que define las operaciones comunes para todos los objetos en la jerarquía, tanto hojas como compuestos.

Inventariocomponent · JAVA

Copiar



```
1 package composite;
2
3 import java.util.List;
4
5 /**
6  * Interfaz Component del patrón Composite.
7  *
8  * Define el contrato común que comparten tanto los nodos hoja (Producto)
9  * como los nodos compuestos (Empresa, Bodega, Estanteria).
10 * El código cliente interactúa únicamente con esta interfaz,
11 * sin importar si el nodo es una hoja o un compuesto.
12 *
13 * Proyecto: Estanterías Inteligentes INC
14 * Patrón: Composite – Módulo Datos/Contenido
15 * Autores: Ana María Bautista Rodríguez (95429)
16 *          Juan Andrés Veintinilla (124596)
17 * USB Cali – Diseño Detallado de Software 2026-1
18 */
19 public interface InventarioComponent {
20
21     /** Nombre descriptivo del nodo (empresa, bodega, estantería o producto). */
22     String getNombre();
23
24     /**
25      * Cantidad total de unidades.
26      * En hojas: retorna el stock directo.
27      * En compuestos: suma recursiva de todos los hijos.
28      */
```



```
29     int getCantidadTotal();
30
31     /**
32      * Valor monetario total en COP.
33      * En hojas: stock * precio unitario.
34      * En compuestos: suma recursiva de todos los hijos.
35      */
36     double getValorTotal();
37
38     /**
39      * Número de productos hoja contenidos en este nodo.
40      * Permite al dashboard mostrar cuántos SKUs distintos maneja
41      * un nivel de la jerarquía.
42      */
43     int contarProductos();
44
45     /**
46      * Double-dispatch hacia el Visitor.
47      * Cada implementación invoca el método visit correspondiente
48      * (visitEmpresa, visitBodega, visitEstanteria, visitProducto).
49      *
50      * @param visitor operación a ejecutar sobre este nodo
51      */
52     void accept(InventarioVisitor visitor);
53
54     /**
55      * Lista de hijos directos.
56      * Las hojas (Producto) retornan una lista vacía.
57      * Los compuestos retornan sus hijos inmediatos.
58      */
```


9.1.1.2 Empresa (Root)

Ahora que se tiene la interfaz común para cada nodo del árbol, empezamos por el nodo raíz (root) del cual los demás se desprenden

Empresa · JAVA

Copiar 

```
1  package composite;
2
3  import java.util.ArrayList;
4  import java.util.Collections;
5  import java.util.List;
6
7  /**
8   * Nodo raíz del patrón Composite.
9   *
10  * Representa al cliente corporativo que contrata el servicio de
11  * Estanterías Inteligentes INC. Gestiona múltiples Bodegas y es
12  * el punto de entrada para cualquier operación transversal
13  * (visitors, reportes, auditorías).
14  *
15  * Corresponde a la entidad gestionada por la API_Empresas del sistema.
16  *
17  * Proyecto: Estanterías Inteligentes INC
18  * Patrón:   Composite – Módulo Datos/Contenido
19  */
20  public class Empresa implements InventarioComponent {
21
22      private final String nombre;
23      private final String nit;
24      private final List<InventarioComponent> bodegas;
25
26      /**
27       * @param nombre nombre legal de la empresa
28       * @param nit     NIT o identificador tributario
29       */
```



```
30     public Empresa(String nombre, String nit) {
31         this.nombre = nombre;
32         this.nit = nit;
33         this.bodegas = new ArrayList<>();
34     }
35
36     // — Gestión de hijos —————
37
38     public void agregarBodega(Bodega bodega) {
39         bodegas.add(bodega);
40     }
41
42     public void quitarBodega(Bodega bodega) {
43         bodegas.remove(bodega);
44     }
45
46     // — InventarioComponent —————
47
48     @Override
49     public String getNombre() {
50         return nombre;
51     }
```



```
52
53     /** Suma recursiva del stock de todas las bodegas de la empresa. */
54     @Override
55     public int getCantidadTotal() {
56         return bodegas.stream()
57             .mapToInt(InventarioComponent::getCantidadTotal)
58             .sum();
59     }
60
61     /** Suma recursiva del valor de todas las bodegas de la empresa. */
62     @Override
63     public double getValorTotal() {
64         return bodegas.stream()
65             .mapToDouble(InventarioComponent::getValorTotal)
66             .sum();
67     }
68
69     /** Total de SKUs distintos en toda la empresa. */
70     @Override
71     public int contarProductos() {
72         return bodegas.stream()
73             .mapToInt(InventarioComponent::contarProductos)
74             .sum();
75     }
```

```
76
77  /**
78   * Punto de entrada principal para los Visitors.
79   * Un único accept() sobre la Empresa propaga la operación
80   * por toda la jerarquía (Bodegas → Estanterías → Productos).
81   */
82  @Override
83  public void accept(InventarioVisitor visitor) {
84      visitor.visitEmpresa(this);
85      for (InventarioComponent hijo : bodegas) {
86          hijo.accept(visitor);
87      }
88  }
89
90  @Override
91  public List<InventarioComponent> getHijos() {
92      return Collections.unmodifiableList(bodegas);
93  }
94
95  // — Getters específicos —————
96
97  public String getNit() { return nit; }
98
99  @Override
100  public String toString() {
101      return String.format("Empresa{nombre='%s', nit='%s', bodegas=%d, productos=%d, valor  
Total=%.0f COP}",
102                          nombre, nit, bodegas.size(), contarProductos(), getValorTotal());
103  }
104 }
```

El patrón de Composite comienza a verse desde la raíz, específicamente en la línea de Código “private final List<InventarioComponent> bodegas;” ya que a partir de ahí, la clase Empresa comienza a almacenar objetos del mismo tipo abstracto que ella implementa, generando así el inicio de la estructura jerárquica del Composite.

```
19  */
20  public class Empresa implements InventarioComponent {
21
22      private final String nombre;
23      private final String nit;
24      private final List<InventarioComponent> bodegas;
25  }
```



```
// — Gestión de hijos —————  
  
public void agregarBodega(Bodega bodega) {  
    bodegas.add(bodega);  
}  
  
public void quitarBodega(Bodega bodega) {  
    bodegas.remove(bodega);  
}
```

9.1.1.3 Bodega (Nodo Compuesto / Rama)

Una vez se tiene definido el inicio del árbol, podemos comenzar a ver los nodos que genera, que siendo nuestro caso, serían: Empresa → Bodega → Estantería → Producto.

Bodega · JAVA

Copiar 

```
1  package composite;
2
3  import java.util.ArrayList;
4  import java.util.Collections;
5  import java.util.List;
6
7  /**
8   * Nodo compuesto de segundo nivel del patrón Composite.
9   *
10  * Representa una instalación física (almacén, dark store, bodega de autopartes)
11  * que agrupa múltiples Estanterías. Corresponde a la entidad gestionada
12  * por la API_Bodega del sistema.
13  *
14  * Proyecto: Estanterías Inteligentes INC
15  * Patrón: Composite – Módulo Datos/Contenido
16  */
17  public class Bodega implements InventarioComponent {
18
19      private final String nombre;
20      private final String ciudad;
21      private final List<InventarioComponent> estanterias;
22
23      /**
24       * @param nombre nombre descriptivo (ej. "Bodega Central Cali")
25       * @param ciudad ciudad donde está ubicada la bodega
26       */
27      public Bodega(String nombre, String ciudad) {
28          this.nombre = nombre;
29          this.ciudad = ciudad;
30          this.estanterias = new ArrayList<>();
31      }
```

```
33 // — Gestión de hijos —————
34
35 public void agregarEstanteria(Estanteria estanteria) {
36     estanterias.add(estanteria);
37 }
38
39 public void quitarEstanteria(Estanteria estanteria) {
40     estanterias.remove(estanteria);
41 }
42
43 // — InventarioComponent —————
44
45 @Override
46 public String getNombre() {
47     return nombre;
48 }
49
50 /** Delega a cada estantería y suma sus stocks. */
51 @Override
52 public int getCantidadTotal() {
53     return estanterias.stream()
54         .mapToInt(InventarioComponent::getCantidadTotal)
55         .sum();
56 }
```



```
58     /** Delega a cada estantería y suma sus valores. */
59     @Override
60     public double getValorTotal() {
61         return estanterias.stream()
62             .mapToDouble(InventarioComponent::getValorTotal)
63             .sum();
64     }
65
66     /** Cuenta todos los productos hoja bajo esta bodega. */
67     @Override
68     public int contarProductos() {
69         return estanterias.stream()
70             .mapToInt(InventarioComponent::contarProductos)
71             .sum();
72     }
73
74     /**
75      * Double-dispatch hacia el visitor, luego propaga accept()
76      * a todas las estanterías hijas.
77      */
78     @Override
79     public void accept(InventarioVisitor visitor) {
80         visitor.visitBodega(this);
81         for (InventarioComponent hijo : estanterias) {
82             hijo.accept(visitor);
83         }
84     }
85
86     @Override
87     public List<InventarioComponent> getHijos() {
88         return Collections.unmodifiableList(estanterias);
89     }
90
91     // — Getters específicos —————
92
93     public String getCiudad() { return ciudad; }
94
95     @Override
96     public String toString() {
97         return String.format("Bodega{nombre='%s', ciudad='%s', estanterias=%d, productos=%d}",
98             nombre, ciudad, estanterias.size(), contarProductos());
99     }
100 }
```


9.1.1.4 Estanteria (Nodo / Rama)

Estanteria · JAVA

Copiar 

```
1 package composite;
2
3 import java.util.ArrayList;
4 import java.util.Collections;
5 import java.util.List;
6
7 /**
8  * Nodo compuesto de primer nivel del patrón Composite.
9  *
10 * Representa una unidad física de almacenamiento dentro de una Bodega,
11 * equipada con sensores IoT (peso, RFID, Pick-to-Light).
12 * Puede contener uno o más Productos (hojas).
13 *
14 * Las operaciones getCantidadTotal(), getValorTotal() y contarProductos()
15 * delegan recursivamente a sus hijos, sin que el código cliente
16 * necesite conocer cuántos productos hay ni cómo están distribuidos.
17 *
18 * Proyecto: Estanterías Inteligentes INC
19 * Patrón: Composite – Módulo Datos/Contenido
20 */
21 public class Estanteria implements InventarioComponent {
22
23     private final String nombre;
24     private final String zona;
25     private final List<InventarioComponent> productos;
```

```
28      * @param nombre nombre identificador de la estantería (ej. "Estantería A-01")
29      * @param zona    zona o categoría física dentro de la bodega (ej. "Farmacia")
30      */
31      public Estanteria(String nombre, String zona) {
32          this.nombre = nombre;
33          this.zona    = zona;
34          this.productos = new ArrayList<>();
35      }
36
37      // — Gestión de hijos —————
38
39      /**
40       * Agrega un producto a esta estantería.
41       * En el sistema real, este método sería invocado cuando un sensor IoT
42       * detecta la colocación de un nuevo artículo.
43       */
44      public void agregarProducto(Producto producto) {
45          productos.add(producto);
46      }
47
48      /** Elimina un producto de la estantería (ej. tras despacho confirmado). */
49      public void quitarProducto(Producto producto) {
50          productos.remove(producto);
51      }
52
```



```
53 // — InventarioComponent —————
54
55 @Override
56 public String getNombre() {
57     return nombre;
58 }
59
60 /** Suma el stock de todos los productos contenidos. */
61 @Override
62 public int getCantidadTotal() {
63     return productos.stream()
64         .mapToInt(InventarioComponent::getCantidadTotal)
65         .sum();
66 }
67
68 /** Suma el valor monetario de todos los productos contenidos. */
69 @Override
70 public double getValorTotal() {
71     return productos.stream()
72         .mapToDouble(InventarioComponent::getValorTotal)
73         .sum();
74 }
75
76 /** Cuenta los productos hoja dentro de esta estantería. */
77 @Override
78 public int contarProductos() {
79     return productos.stream()
80         .mapToInt(InventarioComponent::contarProductos)
81         .sum();
82 }
```

```
83
84     /**
85      * Double-dispatch: notifica al visitor que está procesando una Estanteria,
86      * luego delega accept() a cada hijo para que el visitor los procese también.
87      */
88     @Override
89     public void accept(InventarioVisitor visitor) {
90         visitor.visitEstanteria(this);
91         for (InventarioComponent hijo : productos) {
92             hijo.accept(visitor);
93         }
94     }
95
96     @Override
97     public List<InventarioComponent> getHijos() {
98         return Collections.unmodifiableList(productos);
99     }
100
101     // — Getters específicos —————
102
103     public String getZona() { return zona; }
104
105     @Override
106     public String toString() {
107         return String.format("Estanteria{nombre='%s', zona='%s', productos=%d, stock=%d}",
108             nombre, zona, contarProductos(), getCantidadTotal());
109     }
110 }
```

9.1.1.5 Producto (Leaf / Hoja)

Con esta última pieza del código llegamos al final del árbol, o como se le conoce mejor dentro de la estructura del patrón Composite: “leaf”. Esta parte del código es donde muere la contención de otros componentes, así como la delegación de hijos; Básicamente es el final de la jerarquía. Se puede ver gracias a que retorna una lista vacía.

Producto · JAVA

Copiar



```
1 package composite;
2
3 import java.util.Collections;
4 import java.util.List;
5
6 /**
7  * Hoja del patrón Composite.
8  *
9  * Representa un artículo concreto almacenado en una Estantería.
10 * No tiene hijos: getHijos() retorna una lista vacía.
11 * getCantidadTotal() y getValorTotal() operan directamente sobre
12 * sus propios campos (sin delegación recursiva).
13 *
14 * Proyecto: Estanterías Inteligentes INC
15 * Patrón: Composite – Módulo Datos/Contenido
16 */
17 public class Producto implements InventarioComponent {
18
19     private final String nombre;
20     private final String sku;
21     private final String proveedor;
22     private final double precio;
23     private final double peso;
24     private int stock;
25     private final int stockMinimo;
26 }
```

```
27  /**
28   * @param nombre    nombre descriptivo del producto
29   * @param sku        identificador único de referencia
30   * @param proveedor  nombre del proveedor habitual
31   * @param precio     precio unitario en COP
32   * @param peso       peso en kilogramos por unidad
33   * @param stock      unidades disponibles al momento de crear el objeto
34   * @param stockMinimo umbral mínimo antes de disparar alerta de reabastecimiento
35   */
36  public Producto(String nombre, String sku, String proveedor,
37                  double precio, double peso, int stock, int stockMinimo) {
38      this.nombre    = nombre;
39      this.sku       = sku;
40      this.proveedor = proveedor;
41      this.precio    = precio;
42      this.peso      = peso;
43      this.stock     = stock;
44      this.stockMinimo = stockMinimo;
45  }
46
47  // — InventarioComponent —————
48
49  @Override
50  public String getNombre() {
51      return nombre;
52  }
```

```
47 // — InventarioComponent —————
48
49 @Override
50 public String getNombre() {
51     return nombre;
52 }
53
54 /** En la hoja, la cantidad total ES el stock propio. */
55 @Override
56 public int getCantidadTotal() {
57     return stock;
58 }
59
60 /** Valor = stock × precio unitario. */
61 @Override
62 public double getValorTotal() {
63     return stock * precio;
64 }
65
66 /** Una hoja cuenta como 1 producto. */
67 @Override
68 public int contarProductos() {
69     return 1;
70 }
71
72 /** Double-dispatch: delega al visitor el comportamiento para Producto. */
73 @Override
74 public void accept(InventarioVisitor visitor) {
75     visitor.visitProducto(this);
76 }
```

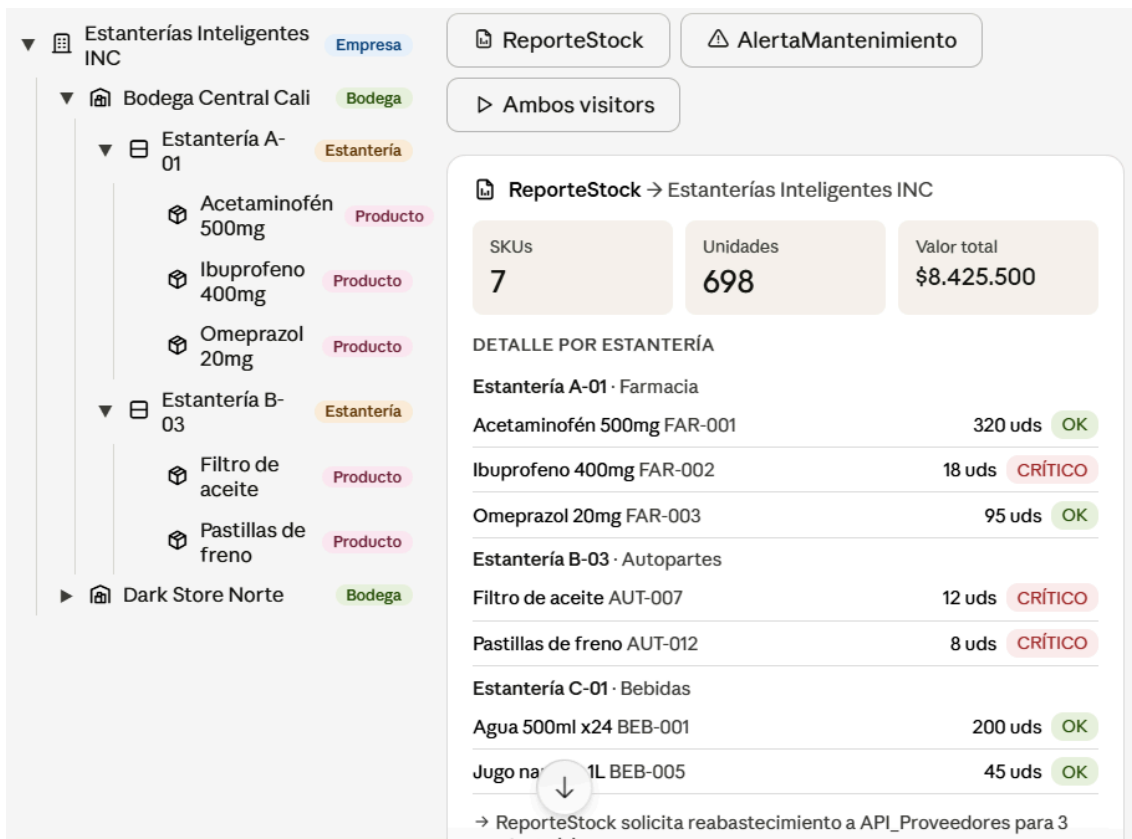
```
78     /** Las hojas no tienen hijos. */
79     @Override
80     public List<InventarioComponent> getHijos() {
81         return Collections.emptyList();
82     }
83
84     // — Getters específicos de Producto —————
85
86     public String getSku()         { return sku; }
87     public String getProveedor()   { return proveedor; }
88     public double getPrecio()      { return precio; }
89     public double getPeso()        { return peso; }
90     public int   getStock()        { return stock; }
91     public int   getStockMinimo()  { return stockMinimo; }
92
93     /** Retorna true si el stock actual está por debajo del mínimo configurado. */
94     public boolean tieneStockCritico() {
95         return stock < stockMinimo;
96     }
97
98     /** Permite actualizar el stock tras una lectura del sensor IoT. */
99     public void actualizarStock(int nuevaCantidad) {
100         if (nuevaCantidad < 0) throw new IllegalArgumentException("Stock no puede ser negati
101 vo.");
102         this.stock = nuevaCantidad;
103     }
104
105     @Override
106     public String toString() {
107         return String.format("Producto{nombre='%s', sku='%s', stock=%d, minimo=%d, precio=%.
108 0f}",
```


9.2 Prototipado con Patron “Visitor”

Nuestro sistema tiene diferentes opciones de operaciones para el inventario del cliente, pero nuestro proyecto con el pasar del tiempo requerirá ejecutar nuevas funciones, tales como por ejemplo: Generar Reporte PDF y Alerta de Mantenimiento, pero debido a la naturaleza de la codificación, implementar cada operación directamente a las clases dentro del dominio del sistema provocaría muchos errores y malos contratiempos a la hora de realizar un producto de calidad.

Por ello, el patrón de Diseño Visitor nos deja Agregar funciones sobre elementos de una estructura de objetos sin cambiar las clases de esos elementos, encapsulando así misma en una clase Visitor Separada.

9.2.1 Solución Prototipo “Visitor”



Esteras Inteligentes INC Empresa

Bodega Central Cali Bodega

Esteras A-01 Esteras

- Acetaminofén 500mg Producto
- Ibuprofeno 400mg Producto
- Omeprazol 20mg Producto

Esteras B-03 Esteras

- Filtro de aceite Producto
- Pastillas de freno Producto

Dark Store Norte Bodega

ReporteStock **AlertaMantenimiento**

Ambos visitors

ReporteStock → Esteras Inteligentes INC

SKUs	Unidades	Valor total
7	698	\$8.425.500

DETALLE POR ESTERAS

Esteras A-01 · Farmacia

Acetaminofén 500mg FAR-001	320 uds	OK
Ibuprofeno 400mg FAR-002	18 uds	CRÍTICO
Omeprazol 20mg FAR-003	95 uds	OK

Esteras B-03 · Autopartes

Filtro de aceite AUT-007	12 uds	CRÍTICO
Pastillas de freno AUT-012	8 uds	CRÍTICO

Esteras C-01 · Bebidas

Agua 500ml x24 BEB-001	200 uds	OK
Jugo naranja 1L BEB-005	45 uds	OK

→ ReporteStock solicita reabastecimiento a API_Proveedores para 3



NODO OBJETIVO

▼ Estanterías Inteligentes INC Empresa

▼ Bodega Central Cali Bodega

▼ Estantería A-01 Estantería

Acetaminofén 500mg Producto

Ibuprofeno 400mg Producto

Omeprazol 20mg Producto

▼ Estantería B-03 Estantería

Filtro de aceite Producto

Pastillas de freno Producto

▶ Dark Store Norte Bodega

EJECUTAR VISITOR

ReporteStock

AlertaMantenimiento

Ambos visitors

AlertaMantenimiento → Estanterías Inteligentes INC

Estanterías	Sensores	Operatividad
3	8	75%

FALLOS DETECTADOS (2)

SENS-A01-RFID-1 · Estantería A-01 FALLO

SENS-B03-RFID-1 · Estantería B-03 SIN_SEÑAL

→ AlertaMantenimiento notifica al equipo técnico sobre 2 sensor(es) con fallo.

ⓘ visitarProducto() no ejecuta ninguna acción en este visitor — solo le interesan los sensores de las estanterías. (SRP)

9.2.1.1 InventarioVisitor (Interfaz)

Inventariovisitor · JAVA

Copiar 

```
1 package visitor;
2
3 import composite.Estanteria;
4 import composite.Producto;
5
6 /**
7  * =====
8  * PATRÓN VISITOR – Interfaz principal
9  * Módulo: Arquitectura
10 * Proyecto: Estanterías Inteligentes INC
11 * =====
12 *
13 * PROBLEMA QUE RESUELVE:
14 * Queremos agregar funcionalidades constantemente (Calculador de Alertas,
15 * Generador de Reportes, etc.) sin modificar las clases Estanteria o
16 * Producto cada vez que se nos ocurra una nueva función.
17 * Esto protege el Principio de Responsabilidad Única (SRP): Estanteria
18 * y Producto solo saben almacenar sus datos, no saben generar reportes
19 * ni revisar sensores.
20 *
21 * SOLUCIÓN:
22 * Esta interfaz define el "contrato" que debe cumplir cualquier nueva
23 * funcionalidad. Cada vez que se quiera agregar una operación nueva,
24 * se crea una clase que implemente este contrato – sin tocar el dominio.
25 *
26 * Autores: Ana María Bautista Rodríguez (95429)
27 *          Juan Andrés Veintinilla (124596)
28 * USB Cali – Diseño Detallado de Software 2026-1
29 */
```

```
30 public interface InventarioVisitor {
31
32     /**
33      * Opera sobre un nodo Producto de la jerarquía.
34      * Cada Visitante Concreto decide qué hace con los datos del producto:
35      * acumularlos para un reporte, verificar su stock, etc.
36      *
37      * @param producto hoja de la jerarquía Composite
38      */
39     void visitarProducto(Producto producto);
40
41     /**
42      * Opera sobre un nodo Estantería de la jerarquía.
43      * Cada Visitante Concreto decide qué hace con la estantería:
44      * revisar sus sensores IoT, calcular su valor total, etc.
45      *
46      * @param estanteria nodo compuesto que contiene productos y sensores IoT
47      */
48     void visitarEstanteria(Estanteria estanteria);
49 }
50
```

En este código el Patrón Visitor crea una Interfaz, donde define un contrato común el cual todas las nuevas funcionalidades deberán seguir. Básicamente dice: Cualquier clase que quiera comportarse como un visitante debe saber trabajar con Producto y Estantería; Todo esto es necesario para agregar nuevas funcionalidades sin modificar Producto y Estantería.

9.2.1.2 Producto (Clase)

Producto · JAVA

Copiar 

```
1 package composite;
2
3 import visitor.InventarioVisitor;
4
5 /**
6  * =====
7  * PATRÓN VISITOR – Elemento Concreto: Producto
8  * Módulo: Arquitectura
9  * Proyecto: Estanterías Inteligentes INC
10 * =====
11 *
12 * Esta clase representa un artículo del inventario.
13 * Su única responsabilidad es almacenar los datos del producto (SRP).
14 *
15 * Gracias al patrón Visitor, esta clase NUNCA necesita ser modificada
16 * cuando se agrega una nueva funcionalidad (reporte, alerta, etc.).
17 * Solo expone accept() para que los visitantes puedan operar sobre ella.
18 */
19 public class Producto {
20
21     private final String nombre;
22     private final String sku;
23     private final String proveedor;
24     private final double precio;
25     private int stock;
26     private final int stockMinimo;
```



```
28     /**
29     * @param nombre    nombre descriptivo del artículo
30     * @param sku       código de referencia único
31     * @param proveedor proveedor habitual del artículo
32     * @param precio    precio unitario en COP
33     * @param stock     unidades disponibles actualmente
34     * @param stockMinimo umbral mínimo antes de generar alerta
35     */
36     public Producto(String nombre, String sku, String proveedor,
37                     double precio, int stock, int stockMinimo) {
38         this.nombre    = nombre;
39         this.sku       = sku;
40         this.proveedor = proveedor;
41         this.precio    = precio;
42         this.stock     = stock;
43         this.stockMinimo = stockMinimo;
44     }
45
46     // — Punto de entrada para el Visitor —————
47
48     /**
49     * Invita al visitor a operar sobre este producto.
50     * El visitor decide qué hace: acumular datos, verificar stock, etc.
51     * Esta clase NO necesita saber qué hace el visitor (SRP cumplido).
52     *
53     * @param visitor cualquier visitante que implemente InventarioVisitor
54     */
55     public void accept(InventarioVisitor visitor) {
56         visitor.visitarProducto(this);
57     }
```



```
59 // — Getters —————
60
61 public String getNombre()    { return nombre; }
62 public String getSku()       { return sku; }
63 public String getProveedor() { return proveedor; }
64 public double getPrecio()    { return precio; }
65 public int    getStock()     { return stock; }
66 public int    getStockMinimo(){ return stockMinimo; }
67
68 /** true si el stock actual está por debajo del mínimo configurado. */
69 public boolean tieneStockCritico() {
70     return stock < stockMinimo;
71 }
72
73 /** Actualiza el stock tras una lectura del sensor IoT de la estantería. */
74 public void actualizarStock(int nuevaCantidad) {
75     if (nuevaCantidad < 0) throw new IllegalArgumentException("Stock no puede ser negati
76 vo.");
77     this.stock = nuevaCantidad;
78 }
79
80 @Override
81 public String toString() {
82     return String.format("Producto{nombre='%s', sku='%s', stock=%d, minimo=%d, precio=%.
83 0f COP}",
84     nombre, sku, stock, stockMinimo, precio);
85 }
```

Este es el código de nuestros productos, pero el patrón de diseño Visitor se encuentra en ésta fracción del código:

```
// — Punto de entrada para el Visitor —————  
  
/**  
 * Invita al visitor a operar sobre este producto.  
 * El visitor decide qué hace: acumular datos, verificar stock, etc.  
 * Esta clase NO necesita saber qué hace el visitor (SRP cumplido).  
 *  
 * @param visitor cualquier visitante que implemente InventarioVisitor  
 */  
public void accept(InventarioVisitor visitor) {  
    visitor.visitarProducto(this);  
}
```

Como los comentarios dentro del código lo mencionan, esta es la entrada para que cualquier nueva función pueda entrar a la clase para usarla pero con la ventaja que mantendrá la integridad de los datos.

9.2.1.3 SensorIoT (Nueva Función)

Sensoriot · JAVA

Copiar 

```
1  package composite;
2
3  /**
4   * Representa un sensor IoT instalado en una Estanteria.
5   *
6   * Cada estantería puede tener múltiples sensores (peso, RFID, temperatura).
7   * El visitante AlertaMantenimiento recorre los sensores de cada estantería
8   * para detectar fallos técnicos sin que la clase Estanteria deba contener
9   * esa lógica de revisión.
10  *
11  * Proyecto: Estanterías Inteligentes INC
12  */
13  public class SensorIoT {
14
15      public enum Tipo { PESO, RFID, PICK_TO_LIGHT, TEMPERATURA }
16      public enum Estado { OPERATIVO, FALLO, SIN_SEÑAL }
17
18      private final String id;
19      private final Tipo tipo;
20      private Estado estado;
21      private double ultimaLectura;
22
23      /**
24       * @param id          identificador único del sensor (ej. "SENS-A01-PESO-1")
25       * @param tipo          tipo de sensor según el enum Tipo
26       * @param estado        estado actual del sensor
27       * @param ultimaLectura último valor registrado por el sensor
28       */
```



```
29     public SensorIoT(String id, Tipo tipo, Estado estado, double ultimaLectura) {
30         this.id          = id;
31         this.tipo         = tipo;
32         this.estado       = estado;
33         this.ultimaLectura = ultimaLectura;
34     }
35
36     public String getId()          { return id; }
37     public Tipo  getTipo()         { return tipo; }
38     public Estado getEstado()      { return estado; }
39     public double getUltimaLectura() { return ultimaLectura; }
40
41     public boolean estaFallando() {
42         return estado == Estado.FALLO || estado == Estado.SIN_SEÑAL;
43     }
44
45     public void actualizarEstado(Estado nuevoEstado) {
46         this.estado = nuevoEstado;
47     }
48
49     @Override
50     public String toString() {
51         return String.format("SensorIoT{id='%s', tipo=%s, estado=%s, lectura=%.2f}",
52             id, tipo, estado, ultimaLectura);
53     }
54 }
```

9.2.1.4 ReporteStock (Nueva Funcionalidad)

Reportestock · JAVA

Copiar 

```
1  package visitor;
2
3  import composite.Estanteria;
4  import composite.Producto;
5
6  import java.util.ArrayList;
7  import java.util.List;
8
9  /**
10   * =====
11   *  VISITANTE CONCRETO #1 – ReporteStock
12   *  Módulo: Arquitectura
13   *  Proyecto: Estanterías Inteligentes INC
14   *  =====
15   *
16   *  PROPÓSITO:
17   *  Recorre toda la jerarquía (estanterías y productos) acumulando
18   *  datos para generar un informe completo del estado del inventario.
19   *
20   *  CLAVE DEL PATRÓN:
21   *  Esta funcionalidad existe COMPLETAMENTE AQUÍ, en esta clase.
22   *  Ni Estanteria ni Producto saben nada de reportes.
23   *  Si mañana el formato del reporte cambia, solo se modifica este archivo.
24   *
25   *  Autores:  Ana María Bautista Rodríguez (95429)
26   *           Juan Andrés Veintinilla (124596)
27   *  USB Cali – Diseño Detallado de Software 2026-1
28   */
```

```
29 public class ReporteStock implements InventarioVisitor {
30
31     // Acumuladores internos del reporte
32     private final StringBuilder reporte = new StringBuilder();
33     private final List<Producto> productosCriticos = new ArrayList<>();
34     private int totalProductos = 0;
35     private int totalUnidades = 0;
36     private double valorTotal = 0.0;
37
38     // — Métodos visit —————
39
40     /**
41      * Al visitar una Estanteria, escribe el encabezado de sección.
42      * Los productos de esa estanteria serán visitados en accept() de Estanteria,
43      * justo después de que este método retorne.
44      */
45     @Override
46     public void visitarEstanteria(Estanteria estanteria) {
47         reporte.append("\n")
48             .append("┌───────────────────────────────────┐\n")
49             .append(String.format("│ Estantería : %-30s|\n", estanteria.getNombre()))
50             .append(String.format("│ Zona      : %-30s|\n", estanteria.getZona()))
51             .append("└───────────────────────────────────┘\n");
52     }
53
54     /**
55      * Al visitar un Producto, acumula sus datos en el reporte
56      * y lo agrega a la lista de críticos si su stock está bajo.
57      */
```

```
58     @Override
59     public void visitarProducto(Producto producto) {
60         totalProductos++;
61         totalUnidades += producto.getStock();
62         valorTotal    += producto.getValorTotal();
63
64         String estadoStock = producto.tieneStockCritico()
65             ? "▲ CRÍTICO"
66             : "✓ OK";
67
68         reporte.append(String.format(
69             " %-28s | SKU: %-10s | Stock: %4d uds | Mín: %3d | Estado: %s\n",
70             producto.getNombre(),
71             producto.getSku(),
72             producto.getStock(),
73             producto.getStockMinimo(),
74             estadoStock
75         ));
76
77         if (producto.tieneStockCritico()) {
78             productosCriticos.add(producto);
79         }
80     }

82     // — Generación del informe final —————
83
84     /**
85      * Retorna el informe completo formateado como texto.
86      * Debe llamarse después de haber pasado este visitor por todos los
87      * nodos de la jerarquía (via estanteria.accept(this)).
88      *
89      * @return string con el reporte completo del inventario
90      */
91     public String generarInforme() {
92         StringBuilder informe = new StringBuilder();
93         informe.append("===== \n");
94         informe.append("          REPORTE DE STOCK – Estanterías INC      \n");
95         informe.append("===== \n");
96         informe.append(reporte);
97         informe.append("\n————— \n");
98         informe.append(String.format(" Total de productos   : %d SKUs distintos\n", totalPro
99 ductos));
100         informe.append(String.format(" Total de unidades    : %d uds\n", totalUnidades));
101         informe.append(String.format(" Valor total         : $%.0f COP\n", valorTotal));
102         informe.append(String.format(" Productos críticos  : %d\n", productosCriticos.size
103 ()));
```



```
103         if (!productosCriticos.isEmpty()) {
104             informe.append("\n ▲ Productos que requieren reabastecimiento:\n");
105             for (Producto p : productosCriticos) {
106                 informe.append(String.format(
107                     "    → %-28s (stock: %d / mínimo: %d)\n",
108                     p.getNombre(), p.getStock(), p.getStockMinimo()
109                 ));
110             }
111         }
112
113         informe.append("===== \n");
114         return informe.toString();
115     }
116
117     // — Getters para uso programático —————
118
119     public int          getTotalProductos()    { return totalProductos; }
120     public int          getTotalUnidades()     { return totalUnidades; }
121     public double       getValorTotal()        { return valorTotal; }
122     public List<Producto> getProductosCriticos() { return productosCriticos; }
123 }
```

10 Reflexión y Evaluación de los Patrones

10.1 Impactos dentro del Sistema

- Tras la creación de los prototipos para la solución de 2 problemas presentados en nuestro sistema, como grupo llegamos a la conclusión que los patrones de diseño han marcado un impacto significativo a la hora de la organización y realización del código.
- Los patrones Composite Pattern y Visitor Pattern mejoraron el desarrollo del sistema al permitir una arquitectura más organizada, escalable y mantenible; El patrón Composite facilitó la representación de una estructura jerárquica dentro de nuestro inventario (empresa, bodegas, estanterías y productos) de manera uniforme, simplificando el manejo recursivo de operaciones como cálculos de stock y valor total; Por otra parte, el patrón Visitor impactó positivamente el desarrollo del sistema al permitir agregar nuevas funcionalidades, como generación de reportes por PDF, alertas de mantenimiento y análisis de inventario, sin modificar las clases principales de productos o estanterías. Esto redujo el acoplamiento, facilitó el mantenimiento del código y permitió extender el sistema de forma más segura y escalable respetando el principio de responsabilidad única.

10.2 Uso de IA para Prototipado

- Para la creación del Prototipado se le pidió se utilizó la herramienta de inteligencia artificial Claude.ai , como apoyo en la generación y validación de los prototipos presentados en este trabajo.

-